Ceiling Effects and Convergence: Null Results for Instruction Repetition in LLM-Agent Pipelines

Hugues Clouâtre
Université de Bordeaux
hclouatre@u-bordeaux.fr

Amadou Diogo Barry
Institut national de la recherche scientifique (INRS)
amadoudiogo.barry@inrs.ca

Abstract

Context: Prompt repetition, the verbatim duplication of an input transforming `<QUERY>` into `<QUERY><QUERY>`, has been shown to improve accuracy for non-reasoning large language models on retrieval and multiple-choice benchmarks. **Objective:** We ask whether applying this analogous pattern to fixed delegate instructions produces similar gains in multi-step agentic pipelines. **Method:** Three pre-registered controlled experiments used Claude Haiku 4.5 delegates (n=5 per condition, temperature 0.5) assigned either a single-copy or a repeated-prompt instruction under blinded binary rubric scoring, totalling 30 sessions and 3,196 messages. **Results:** Experiment 1 (session-ID refactoring, 6 criteria) yielded a non-significant score delta of +0.30 with five of six criteria saturated at 100% in both groups (Fisher's p=1.000). Experiment 2 (tree-sitter scanner evaluation, 7 criteria) produced a complete ceiling effect: all 10 runs scored 7/7 (Mann-Whitney U=12.5, p=1.000). Experiment 3 (Kotlin grammar synthesis, 7 criteria) revealed rubric-runner co-design failure: three of seven criteria scored 0/1 across both groups because the required investigations were absent from the runner prompt. On four reachable criteria, control scored a mean of 2.00/4 and treatment scored 2.40/4 (U=15, p=0.607). Across all three pilot experiments, we detect no effect of prompt repetition on task success. Treatment agents used 30.6% fewer total tokens in Exp1 but 7.2% and 19.9% more in Exp2–3; the direction reverses with session length, as long sessions save turns while short sessions pay pure overhead. **Conclusions:** Criteria requiring investigations absent from the runner prompt are unreachable by both groups and must be resolved before effect sizes are interpretable.

1 Introduction

Leviathan et al. [2025] demonstrated that verbatim repetition of the input, transforming `<QUERY>` into `<QUERY><QUERY>`, improves accuracy for non-reasoning large language models (LLMs) across a wide range of benchmarks. The intervention wins 47 of 70 benchmark-model combinations with zero losses, gaining up to 76 percentage points (pp) on positional retrieval tasks such as NameIndex. The mechanism is efficient: repeated tokens produce no additional output and impose no mandatory latency penalty on most providers.

Multi-step LLM-agent pipelines constitute a structurally different evaluation setting. Agents execute multi-step tool-call loops, accumulate context across hundreds of messages, and produce structured artifacts graded against binary rubrics. Whether applying the analogous repetition to fixed delegate instructions produces similar gains in multi-step agentic pipelines remains untested.

The publicly available AIDev dataset [Li et al., 2026a] captures approximately one million agent-authored pull requests from real-world GitHub repositories; our study complements this observational corpus with a controlled laboratory design that enables causal inference over a single variable (instruction repetition) while holding task, model, and temperature constant.

RQ: Does prompt repetition (the duplicate-query pattern applied to delegate instructions) improve task success rates in multi-step LLM-agent pipelines?

Three pre-registered controlled experiments address that question; it was formulated and registered before any experiment was run [Kitchenham et al., 2002].

This paper contributes a null result across three pre-registered experiments, a formal characterization of *rubric-runner co-design failure* as a distinct evaluation failure mode in which rubric criteria are structurally unreachable from the runner prompt, and an open dataset of session logs, scoring justifications, and label-map timestamps archived at DOI:10.5281/zenodo.19696593 [Clouatre, 2026], enabling replication and secondary analysis. Negative results remain rare in software engineering research despite explicit calls to value them [Paige et al., 2017]. Without published null results, a positive finding risks being generalized to untested settings [Wieringa and Daneva, 2015]; we report this result to constrain that generalization, while acknowledging its pilot scale.

This work addresses methodological challenges in the empirical evaluation of agentic software systems: failure patterns in agentic pipelines (rubric-runner co-design failure), and testing and evaluation of AI agents (pre-registered rubric scoring pipeline).

2 Related Work

Independently, Qian et al. [2026] show that benchmark saturation varies sharply by task: MATH-500 reaches a discriminability score of 0.16 while AIME 2024 reaches 0.74. Ceiling effects suppress treatment signals regardless of intervention strength. Chen et al. [2026] demonstrate, via causal prompt optimization, that near-ceiling queries produce no detectable treatment signal regardless of prompt quality; all gains concentrate on hard queries. This mirrors Experiments 1 and 2: below the discriminability threshold, outcome variance is too low for any intervention to register.

Zheng et al. [2023] establish LLM-as-judge as a practical but validity-limited approach, recommending inter-rater reliability measures, and report over 80% agreement between GPT-4 judgments and human preferences on MT-Bench, but note position bias. Shen et al. [2023] find that single-judge alignment with human raters is candidate-, dimension-, and quality-dependent, with per-system correlation spread of up to 0.5; rubric-based scoring with judge calibration has been proposed to address this variability [Hashemi et al., 2024]. The MSR 2026 Mining Challenge applied AIDev to characterise agent failure taxonomies, task-stratified acceptance, and review effort [Li et al., 2026c,b,d]; none of these observational studies isolates instruction repetition as a controlled pre-registered independent variable. The present experiments use a single Haiku 4.5 judge; inter-rater reliability was validated in Section 3.3. Yu [2026] report that static parallel agent topologies achieve competitive results on SWE-bench Verified under greedy decoding. This supports the delegate design used here. Substantive null results from sound designs are distinct from methodological failures [Sculley et al., 2018]: a sound design delimits intervention scope; a flawed design reveals evaluation infrastructure defects.

3 Experimental Design

3.1 Architecture

All experiments used the Goose agent framework [Agentic AI Foundation (AAIF), 2026]. The orchestrator spawned 10 async delegates (5 control, 5 treatment); treatment delegates received instructions repeated verbatim.

The treatment mirrors the `<QUERY><QUERY>` pattern of Leviathan et al. [2025] applied to the

Code Snippet 1. Agent configuration used across all three experiments.

```
orchestrator:
  model: claude-sonnet-4-6
  provider: gcp_vertex_ai
  temperature: 0.3
  framework: Goose

delegates:
  model: claude-haiku-4-5
  provider: gcp_vertex_ai
  temperature: 0.5
  count_per_experiment: 10  # 5 control, 5 treatment
  extensions: [developer, context7, brave_search]

treatment:
  method: verbatim_repetition  # instructions repeated x2
  control_lengths_chars: [3805, 3399, 3800]  # exp1, exp2, exp3
```

delegate system prompt rather than to a single-turn query (Figure 1). Goose versions 1.25.0 (Exp1, Exp2) and 1.31.1 (Exp3) were used. The provider was Vertex AI for all primary runs, with Amazon Bedrock as a fallback for stream-decode errors in Exp3. The full control instruction template is reproduced in Appendix A.

3.2 Pre-registration Protocol

The evaluation rubric was locked before any delegate was spawned. Group assignments were sealed in a `label-map.json` file before scoring began, with file creation timestamps serving as the pre-registration record, archived in the supplementary dataset [Clouatre, 2026].

A separate Haiku 4.5 instance performed blind scoring: it received only the delegate output JSON and the rubric criteria. No group labels, run metadata, or other outputs were visible during scoring.

3.3 Scoring Validation

To assess the reproducibility of the rubric scores prior to journal submission, a second LLM judge (Claude Sonnet 4.6) re-scored a stratified sample of 5 sessions using the identical rubric prompts employed by the primary judge (Claude Haiku 4.5). The sample included one control and one treatment session from Experiment 1 (FastMCP refactor), one from each condition in Experiment 2 (Tree-sitter abstract syntax tree (AST) scanner), and one treatment session from Experiment 3 (Kotlin grammar). Per-criterion agreement between the two judges was 100% ($\kappa = 1.00$, 33 criteria across 5 sessions). Session-level pass/fail agreement was also 100%, though Cohen's κ was undefined because all sessions were rated as passing the 50% threshold by both judges. The per-criterion κ of 1.00 indicates almost perfect agreement by Landis–Koch standards Landis and Koch [1977]. The Exp2 ceiling effect (all criteria at 100% in every session) contributed to perfect agreement on those 14 criteria, but the two judges also agreed on all 19 non-ceiling criteria in Experiments 1 and 3, suggesting the rubric is sufficiently well-specified to produce reproducible scores.

The inferential test (Mann-Whitney U, two-tailed, α=0.05) and effect size metric (rank-biserial r via [Kerby, 2014]) were pre-specified from Exp2 onward. For Exp1, Fisher's exact test was used post-hoc when the valid-run count fell to n=4 control and n=5 treatment.

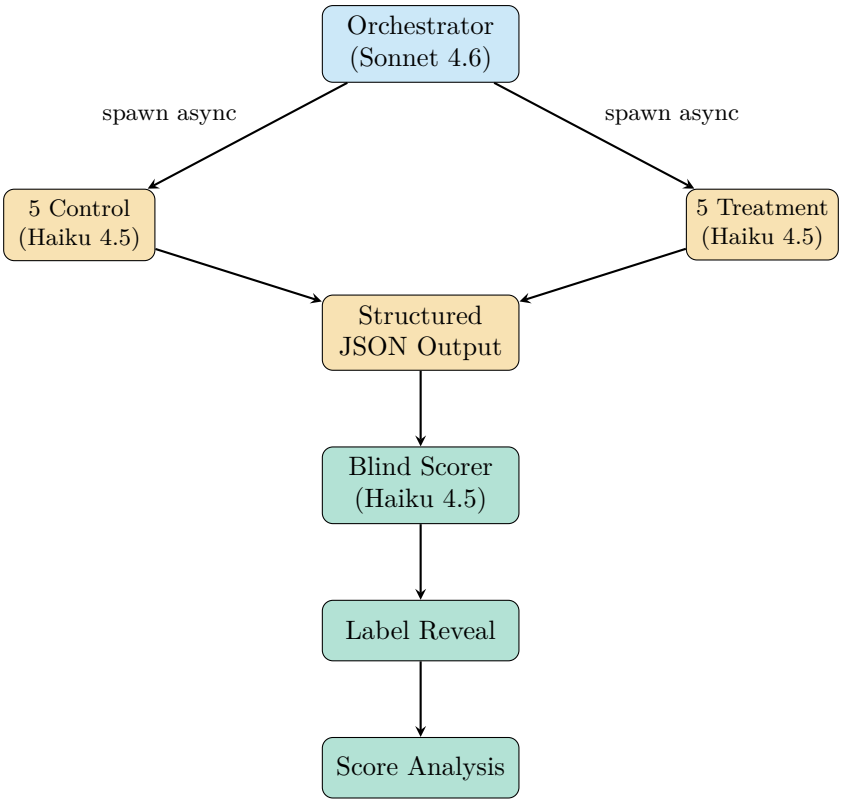


Figure 1: **Blinded evaluation pipeline.** The orchestrator spawns 10 independent delegates per experiment. The blind scorer receives only output JSON and the rubric; group labels are revealed after all scoring is complete.

Statistical power. At n=5 per group, Mann-Whitney U achieves 80% power only for near-complete separation effects ($r \geq 0.80$, equivalent to one group dominating the other by 4:1 or more). Detecting a medium effect (r=0.50) requires $n \approx 20$ per group [Faul et al., 2007]. Infrastructure cost constrained the sample size to 5 runs per group; the experiments are therefore best interpreted as pilot studies capable of detecting large effects or complete separations, not medium effects.

3.4 Experiments

Each experiment targeted an open, unimplemented GitHub issue as the task source, ensuring no published solution existed (see Table 1).

Table 1: **Experiment overview.** Three experiments by task type, rubric width, and valid-run count. Exp1 lost one control run to session drift; Exp3 includes three provider-retry replacements.

Exp	Task	Criteria	Valid runs	Task category
1	FastMCP session ID refactor	6	9	Read-only analysis
2	Tree-sitter AST scanner	7	10	Artifact synthesis (scanner)
3	Kotlin grammar synthesis	7	10	Artifact synthesis (grammar)

Experiment 1 targeted issue `math-mcp-learning-server#222`,[1] a read-only source analysis task: characterise a FastMCP session ID refactor. One control run drifted at 93 messages producing no output; excluded, yielding n=4 valid runs.Experiment 2 targeted `aptu#737`,[2] a code synthesis

[1] https://github.com/clouatre-labs/math-mcp-learning-server/issues/222
[2] https://github.com/clouatre-labs/aptu/issues/737

task evaluating a tree-sitter AST scanner. All 10 runs produced valid output. An undocumented concurrency cap split Exp2 into two batches (see §5.4).

Experiment 3 targeted issue `aptu-coder#649`,[3] a code synthesis task requiring deep inspection of Kotlin grammar application binary interface (ABI) compatibility, node kind taxonomy, and struct fields. Three runs fell back to Bedrock; all produced output and were included.

4 Results

4.1 Pre-analysis Amendments

After scoring was complete, criteria C1, C5, and C6 each scored 0/1 across both groups in Experiment 3. Inspection of the runner prompt (`runner-prompt.md`) confirmed that it contained no instruction directing agents to investigate ABI compatibility between `tree-sitter-kotlin` 0.3.8 and `tree-sitter` 0.26.6 (C1), to write test cases exercising `.kts` file parsing (C5), or to inspect the `LanguageInfo` struct fields or `aptu-coder`#649 to determine whether a `DEFUSE_QUERY` constant was required (C6).

Both groups scored identically (0/1) on C1, C5, and C6, so excluding them cannot bias the treatment comparison in either direction. The exclusion was determined after scoring but before any group labels were inspected, aligning with registered-report amendment practice [Ernst and Baldassarre, 2023]. All three criteria are retained in the full rubric table (see Appendix D); the primary comparison is restricted to C2, C3, C4, and C7.

4.2 Task Success

To address this question, we compare task success rates between control and treatment groups.

Experiment 1 used Fisher's exact test on session-level pass/fail (4/4 vs. 5/5 sessions above the 50% threshold; see §3.3); Mann-Whitney columns are shown for consistency. Experiment 2 produced a complete ceiling effect: all ten runs scored 7/7, rendering the test uninformative (U=12.5, p=1.000, r=n/a).[4] Experiment 3 revealed rubric-runner co-design failure: criteria C1, C5, and C6 scored 0/1 across both groups because the corresponding investigations were absent from the runner prompt. The restricted analysis retaining only criteria C2, C3, C4, and C7 yields identical rank ordering and p-value to the full-rubric analysis (control mean 0.500, treatment mean 0.600, on the 0–1 normalized scale). Across all seven criteria, the full-rubric means are control 0.286 and treatment 0.343; the gap between full-rubric and restricted means confirms a rubric-runner co-design failure rather than a treatment or capability signal.

In Experiment 1, five of six criteria reached 100% pass rate in both groups; only criterion C5 (must-not constraint captured) discriminated between groups: 50% control, 80% treatment (see Table 2). In Experiment 2, every criterion reached 100% in both groups; no discriminating criterion existed. This saturation is consistent with the task-dependent discriminability limits reported by Qian et al. [2026] and the causal analysis of Chen et al. [2026]. In Experiment 3, criterion C7 (language identification) reached 100% in both groups, while criteria C1, C5, and C6 were structurally unreachable (floor). Criterion C3 (ELEMENT_QUERY patterns) achieved 80% across both groups, indicating agents can perform structured synthesis when the task is explicitly set.

[3] `https://github.com/clouatre-labs/aptu-coder/issues/649`; the repository was formerly named `code-analyze-mcp`.

[4] † Effect size undefined; all runs scored identically.

Table 2: **Task success results.** Mann-Whitney U test (two-tailed, α=0.05). Exp1 p-value is from Fisher's exact test (n_{ctrl}=4). † Complete ceiling; test is uninformative. ‡ Degenerate test (n=4 vs. n=5, single discriminating criterion). § Rank-biserial $r = 1 - 2U/(n_1 n_2)$ with U of the treatment group; negative r indicates higher treatment ranks.

Exp	Group	n	Mean score	SD	U	p	r
1 (6 criteria)	Control	4	5.50	0.58	13‡	1.000‡	−0.30§
	Treatment	5	5.80	0.45			
2 (7 criteria)	Control	5	7.00	0.00	12.5†	1.000†	r=n/a
	Treatment	5	7.00	0.00			
3 (4/7 criteria)	Control	5	2.00	0.71	15.0	0.607	−0.20§
	Treatment	5	2.40	0.89			

4.3 Token and Message Efficiency

As a secondary, exploratory outcome measure, we compare token and message counts between conditions.

In Experiment 1, treatment agents consumed 30.6% fewer total tokens than control (median 1,032,311 vs. 716,324); treatment sessions converged in fewer turns (see Table 3). In Experiment 2, treatment agents used 7.2% more total tokens than control (median 703,993 vs. 755,144); sessions were already saturated and no turns were eliminated, so the doubled prompt added net input overhead. In Experiment 3, treatment agents consumed 19.9% more total tokens than control (median 41,880 vs. 50,220); sessions averaged 12 messages with no turns saved, so the doubled prompt imposed pure input overhead with no performance gain.

Table 3: **Resource usage by experiment and condition.** Token counts are median per-run totals. Latency is wall-clock seconds from delegate spawn to output file write. Exp1 control excludes the drift-failure run. Exp3 latency medians exclude runs with provider-retry overhead.

Experiment	Condition	Median Total Tokens	Median Messages	Median Latency (s)
Exp1	Control	1,032,311	204.5	136
	Treatment	716,324	138	93
Exp2	Control	703,993	142	117
	Treatment	755,144	142	112
Exp3	Control	41,880	12	62
	Treatment	50,220	12	72

4.4 Per-Criterion Pass Rates

To characterize the rubric-runner co-design failure identified during analysis (§4.1), we examine per-criterion pass rates.

Figure 2 displays per-criterion pass rates with Wilson 95% confidence intervals. The pattern of near-uniform ceiling saturation in Experiments 1 and 2 contrasts sharply with the bimodal floor-ceiling structure in Experiment 3.

Figure 3 shows token usage by group and experiment. Experiment 3's reversal, where treatment consumed more tokens despite no performance gain, illustrates the cost of repeated prefill in short agentic loops.

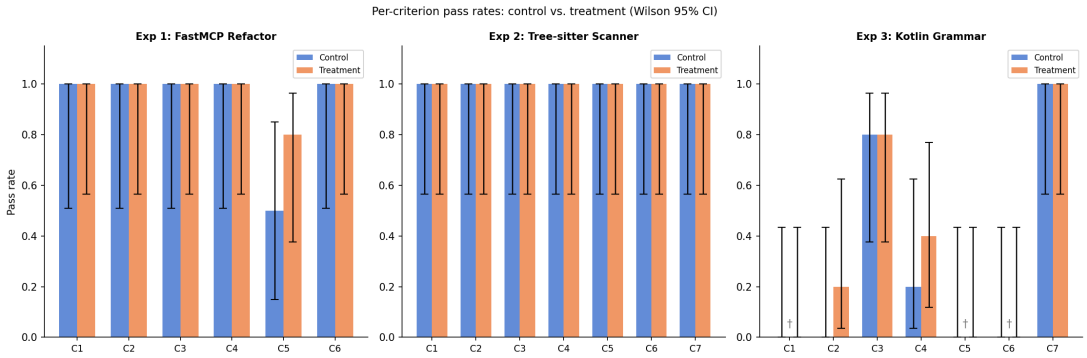


Figure 2: **Per-criterion pass rates.** Control vs. treatment across all three experiments. Wilson 95% confidence intervals shown. Criteria scoring 0/N in both groups are marked with †. (Colors use the Okabe-Ito colorblind-safe palette.)

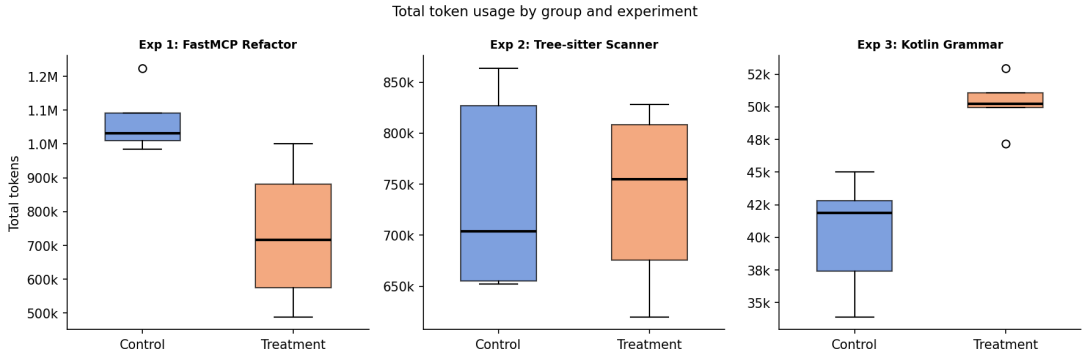


Figure 3: **Token usage by group.** Box plots show median and IQR per experiment. (Colors use the Okabe-Ito colorblind-safe palette.)

5 Discussion

5.1 Mechanism Mismatch

Design choices relative to Leviathan et al. [2025]. Their intervention duplicates the entire user query in single-turn prompts across seven non-reasoning models, selecting benchmarks with substantial headroom below ceiling (e.g., 21% baseline on NameIndex). We deliberately test a different regime: duplication targets the delegate system prompt, tasks are multi-step tool-loop sessions, a single model tier is used for cost control, and tasks were drawn from open, unimplemented issues without baseline-difficulty prescreening. Both calibration failure modes observed here (ceiling in Exp1–2, floor in Exp3) would have been excluded by headroom-first task selection.

Prompt repetition enables each token in `<QUERY>` to attend to every other token in a single causal pass, a mechanism effective only for single-turn inputs where the full context fits in one forward pass; in multi-step loops, the instruction's relative weight dissipates as context grows [Leviathan et al., 2025]. By the time agents in Experiments 1 and 2 had exchanged 140–200 messages, the duplicated instruction constituted a diminishing fraction of total context.

5.2 Why Do the Results Diverge from Leviathan et al.?

Three factors interact. First, mechanism: their gains arise at prefill time because causal attention leaves early query tokens blind to later ones [Leviathan et al., 2025]; duplicating a single-turn query grants every token a second pass over the full context. A system prompt, by contrast, remains fully attended throughout the session, and by messages 140–200 the duplicated block is

a negligible fraction of accumulated context. Second, headroom: their gains concentrate where baseline accuracy is low; our baselines sat at ceiling (Exp1–2) or at a structural floor (Exp3). Third, agents already restate and re-read instructions via the tool-loop context, analogous to the reasoning-model case where repetition adds little.

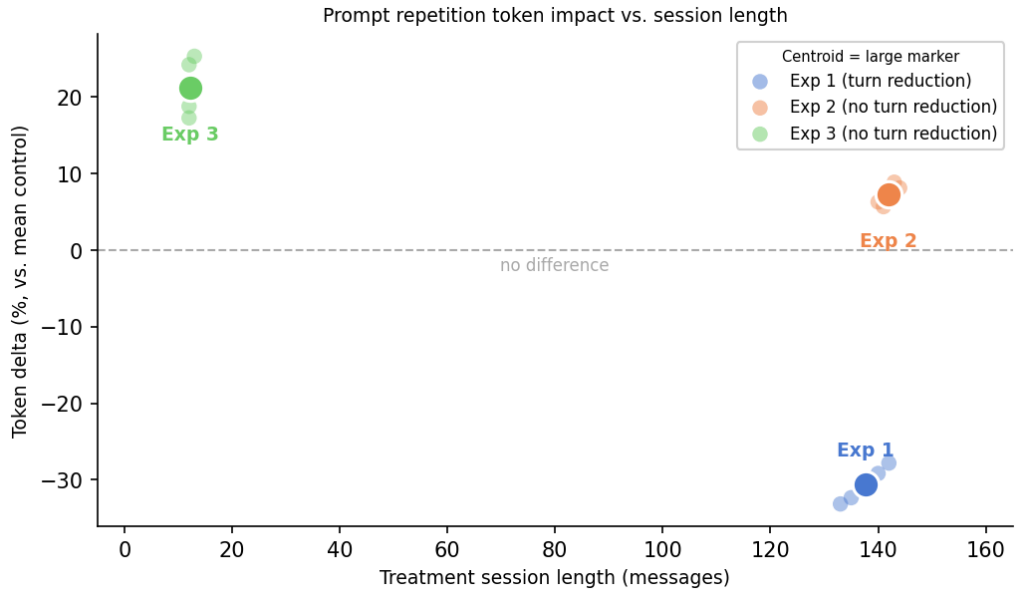


Figure 4: **Prompt repetition token impact vs. session length.** Each point is one experiment; x-axis is mean control session length (messages), y-axis is token delta (%, treatment vs. control), point size scales with mean control token volume. Turn delta is annotated below each point. Token savings require turn reduction; pure overhead accrues when turn count is unchanged. (Colors use the Okabe-Ito colorblind-safe palette.)

The direction of the token delta is therefore a function of session length: in long-session experiments, turn savings dominate; in short-session experiments, prompt overhead dominates. (Figure 4).

Let p be the prompt token count and $\bar{c}$ the mean tokens per message. A repeated prompt adds p tokens to every prefill. Each avoided turn eliminates $\bar{c}$ tokens of accumulated context, where $\bar{c}$ grows with session length. The crossover point L^* satisfies $p = \bar{c}(L^* - 1)$, giving $L^* \approx p/\bar{c}+1$: sessions longer than L^* yield a net token saving; sessions shorter than L^* incur overhead. For Experiment 3 ($p \approx 950$, $\bar{c} \approx 3{,}300$ tokens/message), $L^* \approx 1.3$ messages, well below the observed 12-message median. Yet treatment agents still consumed more tokens, because repeated instructions did not reduce turn count: agents could not complete structurally unreachable criteria regardless of instruction emphasis, so the turn-count assumption underlying L^* does not hold under rubric-runner misalignment.

5.3 Rubric-Runner Co-design Failure

A criterion is *structurally unreachable* if no instruction in the runner prompt tasks the corresponding investigation (defined in §4.1). The resulting floor effect is not an agent signal; it is an evaluation design artifact. Experiment 3 exemplifies this failure mode. Criteria C1, C5, and C6 required agents to inspect ABI compatibility constraints, node-kind taxonomy sources, and struct field layouts; none of these investigations appeared in the runner prompt, and both groups scored 0/1 on all three criteria.

We propose a rubric-runner alignment score $A = k_r/k$ where k is the total number of rubric criteria and k_r is the number of criteria traceable to at least one explicit instruction in the runner

prompt. For Experiment 3, A=4/7=0.57; for Experiments 1 and 2, A=1.0. We recommend A=1.0 as a necessary (though not sufficient) pre-registration gate: any criterion with A<1.0 should be revised or removed before sealing the rubric. This eliminates a class of evaluation failures endemic to LLM-agent pipelines where rubric authors and prompt authors operate independently.

Midolo et al. [2026] derive ten prompt improvement guidelines for code generation via iterative test-driven refinement, finding that explicitly specifying output post-conditions and I/O contracts produces the largest gains in test-pass rates, while structural repetition does not appear among the improvement patterns. This corroborates the rubric-runner alignment principle advanced here: traceability, not repetition count, is the operative lever, as diagnosed post hoc in Experiment 3.

5.4 Infrastructure Confounds

The Goose agent framework enforces a concurrency cap via `GOOSE_MAX_BACKGROUND_TASKS` (defaults to 5; configurable since Goose 1.29[5]). In Experiment 2, this cap silently serialized ten delegates into two batches approximately four minutes apart, introducing a temporal gap not anticipated in the pre-registration. This serialization does not affect the statistical conclusion, but it illustrates a broader risk. Researchers should explicitly log concurrency caps and batch boundaries; undocumented serialization can bias latency measurements and, in non-ceiling experiments, interact with task difficulty.

5.5 Future Work

First, n=5 per group is underpowered for medium effects; replication with $n \geq 20$ and tasks whose baseline pass rate falls below 0.5 would determine whether prompt repetition produces any signal once ceiling and floor artifacts are removed. Second, multi-judge scoring with a reported intraclass correlation coefficient remains a key validity gap. A single LLM judge cannot detect systematic scorer bias; a human–model cross-validation would strengthen internal validity. Third, all delegates used Haiku 4.5; other tiers may exhibit different context-weighting dynamics under prompt repetition.

6 Threats to Validity

6.1 Construct Validity

Three aspects of the rubric design threaten construct validity. First, Section 5.3 documents a rubric-runner co-design failure: three of seven criteria in Experiment 3 scored 0/1 because the runner prompt lacked instructions for the required investigations. As a result, those criteria measure runner-prompt completeness rather than agent capability, limiting our ability to conclude that prompt repetition affects criterion-level pass rates on structurally unreachable items. Second, the binary pass/fail granularity of each criterion (chosen to enable blinded automated scoring across 30 sessions; polytomous scales would require judge calibration beyond the project's budget) compresses within-condition variance, reducing sensitivity to partial improvements that a polytomous or continuous scoring scheme might detect. Third, scoring stability across alternative rubric phrasings was not tested; a replication with criteria rewritten for full rubric-runner alignment (A=1.0, §5) would test whether the observed effects hold under corrected instructions.

[5] https://github.com/aaif-goose/goose/pull/7940

6.2 Internal Validity

Several uncontrolled sources of variance limit internal validity. First, Goose version 1.25.0 was used for Experiments 1 and 2 while version 1.31.1 was used for Experiment 3; version-specific behavioral changes in agent scheduling or tool-handling logic cannot be ruled out as confounds. Second, three individual runs in Experiment 3 fell back to Bedrock due to stream-decode errors and Vertex AI rate limits before retrying successfully on Vertex; all token and score data derive from the successful Vertex runs, but the transient provider switch cannot be ruled out as a minor infrastructure variable. Third, a single LLM judge (Claude Haiku 4.5; justified by the post hoc two-judge validation, κ=1.00, §3.3; a human–model cross-validation remains future work) performed all rubric scoring with no human calibration; an inter-rater reliability check using a second LLM judge (Claude Sonnet 4.6) across 5 stratified sessions found perfect agreement ($\kappa = 1.00$, see §3.3), though the small sample and ceiling effects limit the precision of this estimate, and our ability to conclude that scores are reproducible across non-LLM judges remains limited. Fourth, task ordering within each session was not randomized or counterbalanced. Order effects such as fatigue, learning, or context carry-over are therefore uncontrolled, limiting our ability to attribute score differences solely to the treatment. Fifth, as discussed in Section 5.4, the `GOOSE_MAX_BACKGROUND_TASKS` concurrency cap serialized Experiment 2 delegates into two batches; while this did not affect the statistical conclusion in the present ceiling-affected data, it limits our ability to conclude that latency or delegation dynamics were uniform across the experiment.

6.3 External Validity

The experimental design constrains external validity along four dimensions. First, only three software-engineering tasks were sampled (session-ID refactoring, tree-sitter scanner implementation, and Kotlin grammar synthesis), which limits our ability to generalize to the broader distribution of SWE-bench-style tasks or to non-engineering domains such as data analysis or creative writing. Second, all experiments used a single model (Claude Haiku 4.5) at a single temperature (0.5; chosen as a compromise between eliminating within-condition variance at temperature 0 and inflating task-failure noise at n=5); results may not generalize to reasoning models, larger tiers, or other providers. Third, a single agentic framework (Goose) was used; framework-specific features such as delegate spawning semantics or context-window boundaries may interact with prompt-repetition effects in ways that limit generalizability to other frameworks such as LangGraph or CrewAI. Fourth, the lab setting used pre-defined tasks, isolated repositories, and scripted pipeline execution. This limits our ability to conclude that the null results hold in production pipelines, which face variable task difficulty, real-time API failures, and human-in-the-loop interruptions.

6.4 Conclusion Validity

Statistical power and design choices constrain conclusion validity. The sample size of n=5 per condition was underpowered for medium effect sizes (Cohen's $d \approx 0.5$); the study was powered for large effects only, which limits our ability to conclude that prompt repetition has no practically meaningful effect at smaller effect sizes. No multiple-comparison correction was applied across the three experiments, inflating the family-wise error rate. Ceiling effects in Experiments 1 and 2 compress variance and further reduce statistical power, making true effects harder to detect even if they exist. Finally, Experiment 1 was not fully pre-registered: the statistical test was not pre-specified. This partial pre-registration mitigates but does not eliminate the risk of HARKing (hypothesizing after results are known), and limits our ability to classify all three experiments as confirmatory.

7 Conclusion

Across three pre-registered experiments, we find no evidence that prompt repetition improves task-success rates for LLM agents executing structured software-engineering tasks in multi-step pipelines. The null results hold across two distinct calibration failure modes: near-ceiling task difficulty (Experiments 1 and 2) and rubric-runner co-design failure (Experiment 3). The evidence points instead to rubric-runner alignment as the higher-leverage design investment (observed, not experimentally manipulated, in this study). These findings do not extend to the positional-retrieval and multiple-choice settings where prompt repetition's gains are established.

Statements and Declarations

Competing interests. The authors declare no competing interests.

Funding. The authors received no financial support for the research, authorship, and/or publication of this article.

Data availability. All session logs, scoring records, and pre-registration materials are publicly archived at DOI:10.5281/zenodo.19696593 [Clouatre, 2026]. A preprint of this manuscript is archived at DOI:10.5281/zenodo.20039271.

References

Agentic AI Foundation (AAIF). Goose: An open-source ai agent framework. https://github.com/aaif-goose/goose, 2026. No DOI available; open-source software framework.

W. Chen, Y. Fang, S. Fu, F. Xu, and X. Wei. Optimizing prompts for large language models: A causal approach. *arXiv preprint arXiv:2602.01711*, 2026. doi: 10.48550/arXiv.2602.01711. URL https://arxiv.org/abs/2602.01711.

H. Clouatre. Prompt repetition experiments dataset. Zenodo, 2026. URL https://doi.org/10.5281/zenodo.19696593.

N. A. Ernst and M. T. Baldassarre. Registered reports in software engineering. *Empirical Software Engineering*, 28:55, 2023. doi: 10.1007/s10664-022-10277-5.

F. Faul, E. Erdfelder, A.-G. Lang, and A. Buchner. G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behavior Research Methods*, 39(2):175–191, 2007. doi: 10.3758/BF03193146.

H. Hashemi, J. Eisner, C. Rosset, B. Van Durme, and C. Kedzie. LLM-rubric: A multidimensional, calibrated approach to automatic evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.acl-long.745. URL https://aclanthology.org/2024.acl-long.745.

D. S. Kerby. The simple difference formula: An approach to teaching nonparametric correlation. *Comprehensive Psychology*, 3:11.IT.3.1, 2014. doi: 10.2466/11.IT.3.1.

B. A. Kitchenham, S. L. Pfleeger, L. M. Pickard, P. W. Jones, D. C. Hoaglin, K. El Emam, and J. Rosenberg. Preliminary guidelines for empirical research in software engineering. *IEEE Transactions on Software Engineering*, 28(8):721–734, 2002. doi: 10.1109/TSE.2002.1027796.

J. R. Landis and G. G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.

Y. Leviathan, M. Kalman, and Y. Matias. Prompt repetition improves non-reasoning LLMs. *arXiv preprint arXiv:2512.14982*, 2025. doi: 10.48550/arXiv.2512.14982. URL https://arxiv.org/abs/2512.14982.

H. Li, H. Zhang, and A. E. Hassan. AIDev: Studying AI coding agents on GitHub, 2026a. URL https://arxiv.org/abs/2602.09185.

H. Li et al. Comparing AI coding agents: A task-stratified analysis of pull request acceptance, 2026b. URL https://arxiv.org/abs/2602.08915.

H. Li et al. Where do AI coding agents fail? An empirical study of failed agentic pull requests, 2026c. URL https://arxiv.org/abs/2601.15195.

H. Li et al. Early-stage prediction of review effort in AI-generated pull requests, 2026d. URL https://arxiv.org/abs/2601.00753.

A. Midolo, A. Giagnorio, F. Zampetti, R. Tufano, G. Bavota, and M. Di Penta. Guidelines to prompt large language models for code generation: An empirical characterization. *arXiv preprint arXiv:2601.13118*, 2026. doi: 10.48550/arXiv.2601.13118. URL https://arxiv.org/abs/2601.13118.

R. F. Paige, J. Cabot, and N. A. Ernst. Foreword to the special section on negative results in software engineering. *Empirical Software Engineering*, 22:2453–2456, 2017. doi: 10.1007/s10664-017-9498-0.

Q. Qian, C. Huang, J. Xu, et al. Benchmark^2: Systematic evaluation of LLM benchmarks. *arXiv preprint arXiv:2601.03986*, 2026. doi: 10.48550/arXiv.2601.03986. URL https://arxiv.org/abs/2601.03986.

D. Sculley et al. Winner's curse? on pace, progress, and empirical rigor. In *ICLR 2018 Workshop: Critiquing and Correcting Trends in Machine Learning*, 2018. URL https://openreview.net/forum?id=rJWF0Fywf.

C. Shen, L. Cheng, X.-P. Nguyen, Y. You, and L. Bing. Large language models are not yet human-level evaluators for abstractive summarization. In *Findings of EMNLP*, 2023. doi: 10.18653/v1/2023.findings-emnlp.278.

R. Wieringa and M. Daneva. Six strategies for generalizing software engineering theories. *Science of Computer Programming*, 101:136–152, 2015. doi: 10.1016/j.scico.2014.11.013.

G. Yu. AdaptOrch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence. *arXiv preprint arXiv:2602.16873*, 2026. doi: 10.48550/arXiv.2602.16873. URL https://arxiv.org/abs/2602.16873.

L. Zheng, W.-L. Chiang, Y. Sheng, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. doi: 10.48550/arXiv.2306.05685. URL https://arxiv.org/abs/2306.05685.

Appendices

A Control Prompt Template

The following listing shows the control instruction template, the Scout delegate prompt used in all three experiments. The treatment condition repeats this block verbatim a second time.

Code Snippet 2. Control prompt template. The treatment condition appends an identical copy immediately below. Sourced from `experiments/exp*/runner-prompt.md`.

```
# Scout Research Agent

You are a Scout research agent. Your role is to explore the codebase,
gather evidence, and produce a structured JSON research report.

## Instructions

1. Clone or inspect the target repository at the specified commit.
2. Read the issue description in full.
3. Investigate all files relevant to the issue.
4. Consult external documentation (Context7, brave_search) as needed.
5. Write a structured JSON report to the output file specified.

## Output Contract

Write your findings to the path provided by the orchestrator.
The output must be valid JSON matching the rubric schema.
Do NOT summarize; include verbatim excerpts and file paths.

## Extensions Available

- developer: file read/write, shell execution
- context7: library documentation lookup
- brave_search: web search

## Constraints

- Temperature: 0.5
- Model: claude-haiku-4-5
- Provider: Vertex AI (Bedrock fallback on stream-decode error)
- Do not modify any source files.
- Do not open pull requests.
```

B Treatment Condition

The treatment appends a verbatim copy of Appendix A with no separator.

Code Snippet 3. Schematic of treatment condition: control instructions repeated verbatim a second time.

```
[INSTRUCTIONS]
[INSTRUCTIONS]  <- verbatim repeat
```

C Per-Run Scores

Table 4 lists all 30 runs across the three experiments with individual scores.

Table 4: **All 30 per-run scores.** Exp1 control-1 excluded from analysis (drift failure). Exp3 C1, C5, C6 scored 0/1 in both groups (rubric-runner co-design failure).

Run ID	Exp	Condition	Score	Criteria	Notes
scout-control-1	1	Control	—	0	Drift failure; no output
scout-control-2	1	Control	6/6	6	
scout-control-3	1	Control	6/6	6	
scout-control-4	1	Control	5/6	6	
scout-control-5	1	Control	5/6	6	
scout-treatment-1	1	Treatment	6/6	6	
scout-treatment-2	1	Treatment	5/6	6	
scout-treatment-3	1	Treatment	6/6	6	
scout-treatment-4	1	Treatment	6/6	6	
scout-treatment-5	1	Treatment	6/6	6	
scout-run-01	2	Control	7/7	7	
scout-run-02	2	Treatment	7/7	7	
scout-run-03	2	Control	7/7	7	
scout-run-04	2	Treatment	7/7	7	
scout-run-05	2	Control	7/7	7	
scout-run-06	2	Treatment	7/7	7	
scout-run-07	2	Control	7/7	7	
scout-run-08	2	Treatment	7/7	7	
scout-run-09	2	Control	7/7	7	
scout-run-10	2	Treatment	7/7	7	
scout-run-01	3	Control	1/7	7	Logging gap
scout-run-02	3	Treatment	2/7	7	
scout-run-03	3	Control	3/7	7	
scout-run-04	3	Treatment	2/7	7	Bedrock fallback
scout-run-05	3	Control	2/7	7	
scout-run-06	3	Treatment	4/7	7	
scout-run-07	3	Control	2/7	7	
scout-run-08	3	Treatment	2/7	7	
scout-run-09	3	Control	2/7	7	Bedrock fallback
scout-run-10	3	Treatment	2/7	7	Bedrock fallback; 780s wall time

D Rubric Criteria

Experiment 1: FastMCP Session ID Refactor (6 criteria)

1. **C1** Source file 1 identified (`src/math_mcp/tools/persistence.py`)
2. **C2** Source file 2 identified (`src/math_mcp/tools/calculate.py`)
3. **C3** Anti-pattern found (`id(ctx.lifespan_context)`)
4. **C4** Replacement API correct (`ctx.set_state` / `ctx.get_state` with UUID)
5. **C5** Must-Not constraint captured (non-serializable values, process-restart caveat)
6. **C6** FastMCP docs consulted (`gofastmcp.com/servers/context` or equivalent)

Experiment 2: Tree-sitter AST Scanner (7 criteria)

1. **C1** SecurityScanner implementation file identified; verified against actual repo structure, not assumed.
2. **C2** Line-by-line regex limitation understood; must reference single-line constraint; cite `aptu#735` or `aptu#736` or quote the test.
3. **C3** tree-sitter-rust version verified against Cargo.toml or Context7 docs; must identify 0.23 (not assumed from issue text alone).
4. **C4** Hybrid vs. full-migration tradeoff articulated with codebase evidence; must name specific patterns or files.
5. **C5** At least 2 specific patterns identified as requiring multi-line detection; must name actual pattern IDs or descriptions from source code.
6. **C6** Data-flow/taint tracking gap noted as unsolved by tree-sitter alone; explicit statement that AST traversal does not equal taint analysis.
7. **C7** Binary size / grammar crate count estimated with specifics; must name at least 3 target languages with their crate names.

Experiment 3: Kotlin Grammar Synthesis (7 criteria)

1. **C1**† tree-sitter-kotlin 0.3.8 LANGUAGE export verified and ABI relationship with tree-sitter 0.26.6 fully characterized with evidence; incompatibility is a valid finding if documented.
2. **C2** Kotlin companion object node kind identified as `object_declaration` with companion modifier; `delegation_specifiers` children enumerated distinguishing `superclass_type_with_constructor` from `user_type`.
3. **C3** At least 3 query patterns from the tree-sitter-kotlin corpus correctly captured in `ELEMENT_QUERY`.
4. **C4** `extract_inheritance` handler correctly walks `delegation_specifiers` and separates nodes of kind `superclass_type_with_constructor` (has argument parens) from `user_type` (no parens).
5. **C5**† Unit tests confirm `.kt` AND `.kts` file parsing both work; at least 1 test with `.kts` syntax.
6. **C6**† `DEFUSE_QUERY` constant created for Kotlin if required by current `LanguageInfo` struct, or justified as None if not applicable; must cite inspection of `LanguageInfo` struct fields or `aptu-coder#649`.
7. **C7** All structural wiring described: feature flag, query constant names, `EXTENSION_MAP` entries, `mod.rs` arms, module registered.

† Excluded from primary treatment comparison (rubric-runner co-design failure; both groups scored 0/1; see §4.1).